



COL7378: Principles of Autonomous Systems

Semester II, 2025-26

Reinforcement Learning: Introduction

Rohan Paul

Outline

- Last Class
 - Markov Decision Processes
- This Class
 - An introduction to Reinforcement Learning
- Reference Material
 - Please follow the notes as the primary reference on this topic.

Acknowledgements

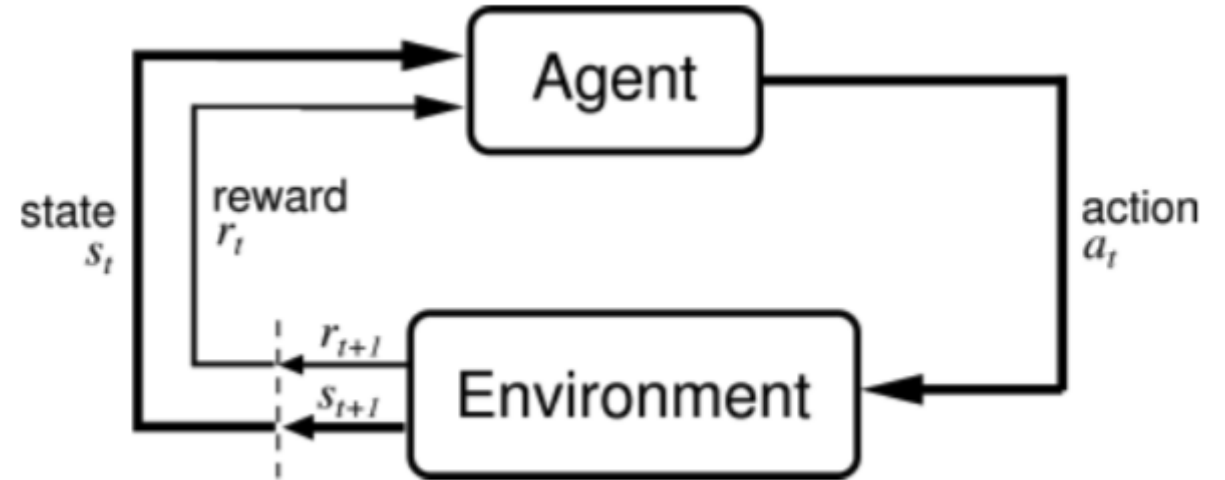
These slides are intended for teaching purposes only. Some material has been used/adapted from web sources and from slides by Nicholas Roy, Wolfram Burgard, Dieter Fox, Sebastian Thrun, Siddharth Srinivasa, Dan Klein, Pieter Abbeel, Max Likhachev and others.

Learning to Act from Interaction/Data

- So far, we assumed to have an a-priori model of the domain
 - MDP: Tuple of states, actions, transition function, rewards, start state and discount factor
 - Safe to assume that some parts of the model are constant.
 - State and action sets are fixed and are given by the domain.
- In practice, we don't know the
 - Transition function (*if I take an action which state I will end up in?*)
 - Reward function (*when I take a transition is it good or bad?*)
- How to tackle this?
 - Interaction and gaining experience
 - We can take actions, observe rewards and which states the agent lands up.
 - Use this data to learn goal-directed actions (implicitly/explicitly understand the missing portions of the MDP).

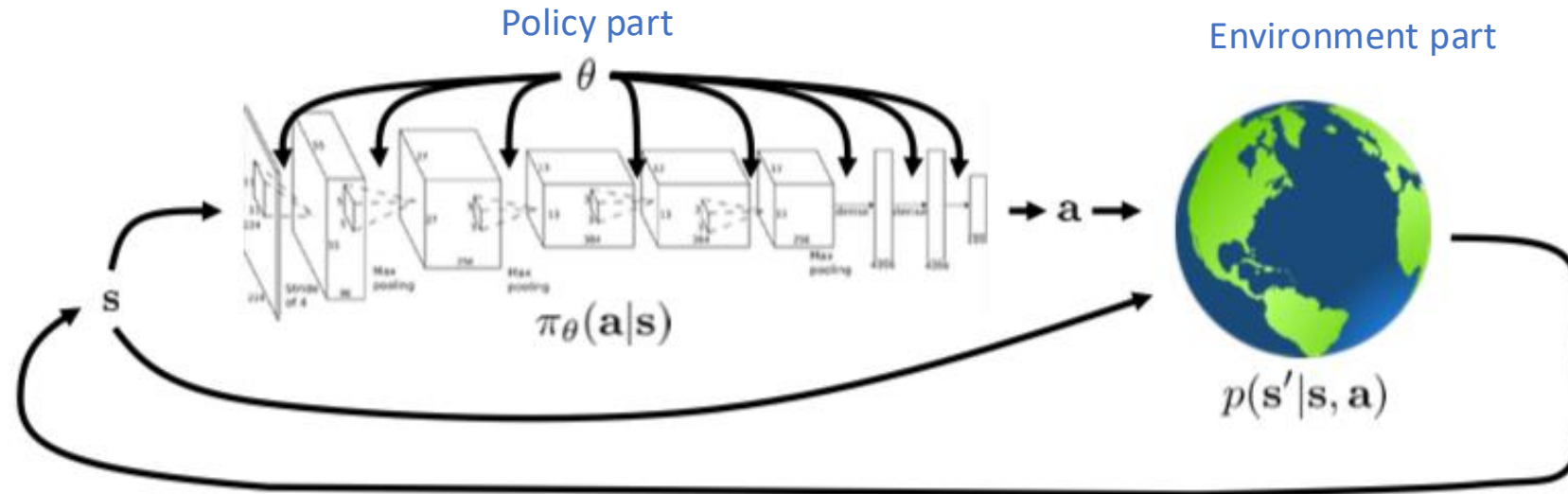
Learning to Act from Data

- Setting
 - The agent experiences the environment and receives a reward and observes the consequence of the action.
 - Does not have the full rewards function or the transition model ahead of time.
- Needs to determine how to act?
 - There is evaluative feedback for actions not prescriptive feedback.



$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^H \gamma^t R_t(S_t, A_t, S_{t+1}) | \pi \right]$$

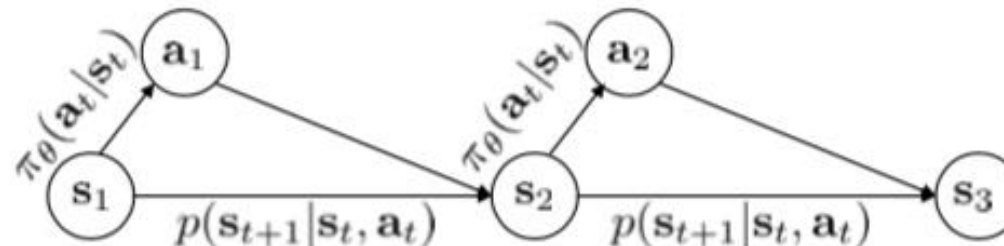
Goal of Reinforcement Learning



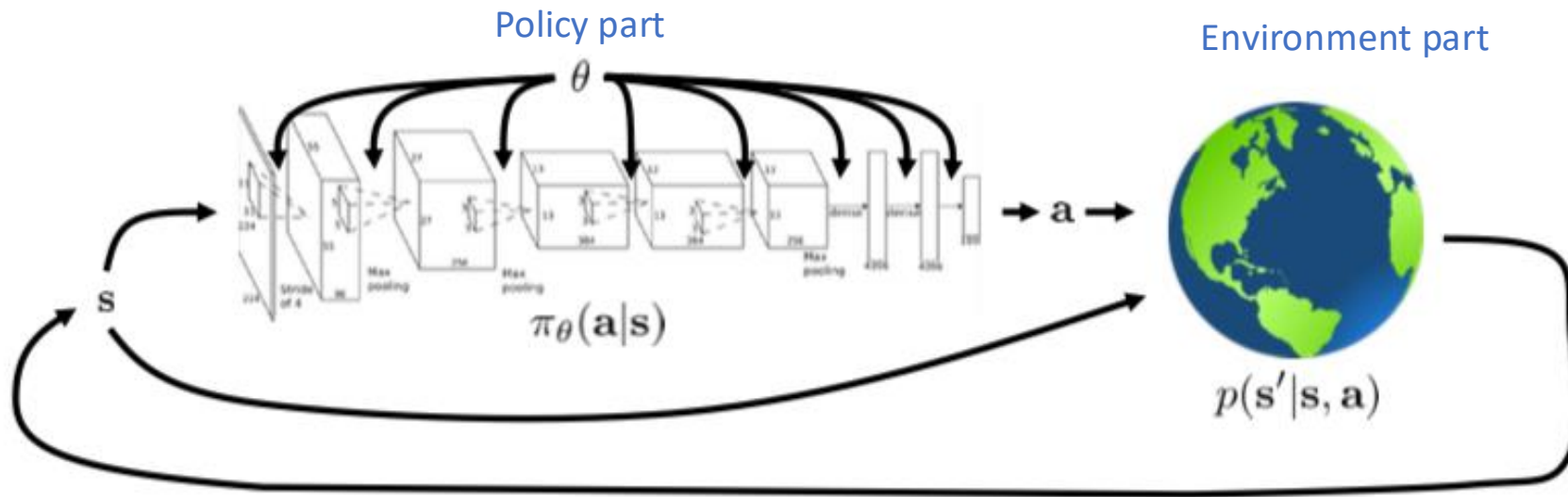
Probability of a state-action trajectory.

$$\underbrace{p_{\theta}(s_1, a_1, \dots, s_T, a_T)}_{p_{\theta}(\tau)} = p(s_1) \prod_{t=1}^T \underbrace{\pi_{\theta}(a_t|s_t)p(s_{t+1}|s_t, a_t)}_{\text{Markov chain on } (s, a)}$$

Expressing as a probabilistic graphical model



Goal of Reinforcement Learning



$$\underbrace{p_{\theta}(s_1, a_1, \dots, s_T, a_T)}_{p_{\theta}(\tau)} = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

What we learn is the policy (with parameter theta) – maximizes an objective.

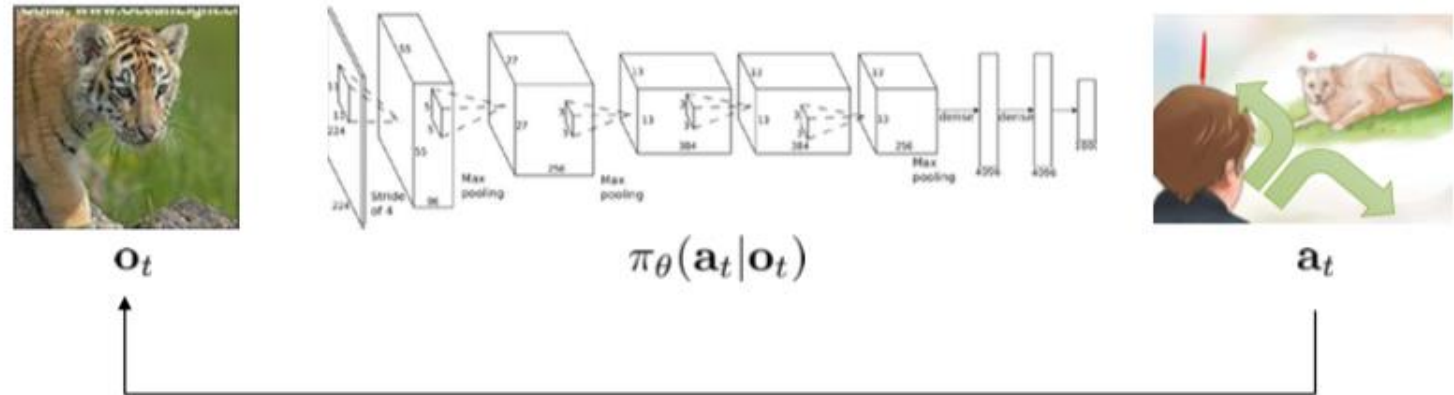
$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right]$$

State Vs. Observation of the State

Decision making should be on the state of the system.

Often the true state is only observed via some form of sensing.

Slight distinction between a policy mapping $s \rightarrow a$ and $o \rightarrow a$



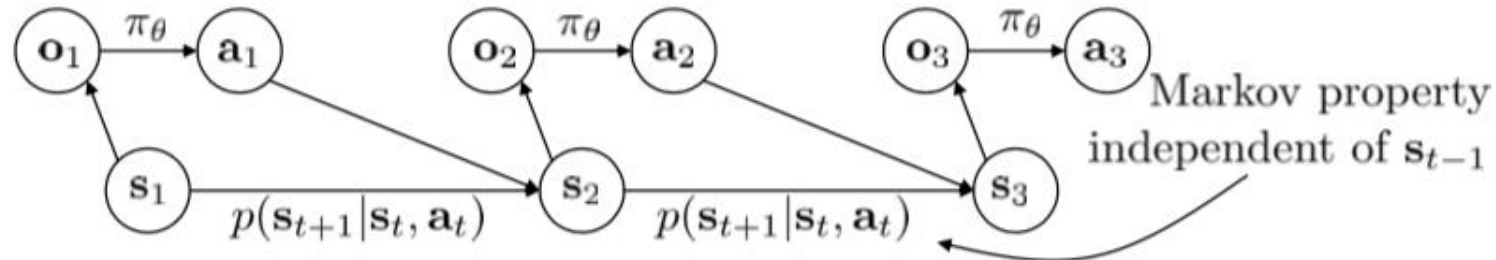
$\mathbf{s}_t$ – state

$\mathbf{o}_t$ – observation

$\mathbf{a}_t$ – action

$\pi_\theta(\mathbf{a}_t | \mathbf{o}_t)$ – policy

$\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)$ – policy (fully observed)



Note: The decision is based on the observation of the state and not the state directly.

Formalism: *MDP* vs. Partially-Observed MDP

Markov decision process

$$\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{T}, r\}$$

$\mathcal{S}$ – state space

states $s \in \mathcal{S}$ (discrete or continuous)

$\mathcal{A}$ – action space

actions $a \in \mathcal{A}$ (discrete or continuous)

$\mathcal{T}$ – transition operator (now a tensor!)

r – reward function

$$r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

$r(s_t, a_t)$ – reward

Formalism: MDP vs. *Partially-Observed MDP*

partially observed Markov decision process $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{E}, r\}$

$\mathcal{S}$ – state space states $s \in \mathcal{S}$ (discrete or continuous)

$\mathcal{A}$ – action space actions $a \in \mathcal{A}$ (discrete or continuous)

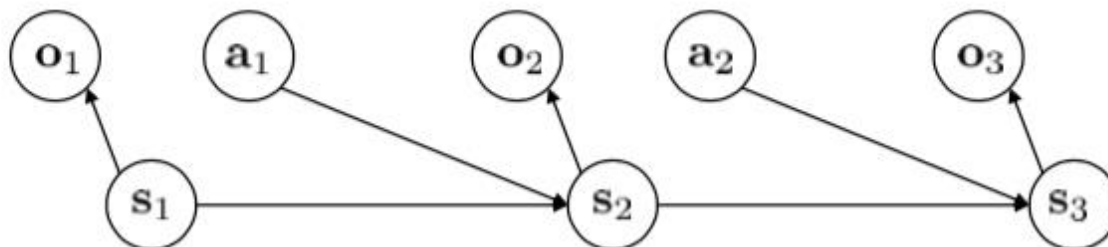
$\mathcal{O}$ – observation space observations $o \in \mathcal{O}$ (discrete or continuous)

$\mathcal{T}$ – transition operator (like before)

$\mathcal{E}$ – emission probability $p(o_t | s_t)$

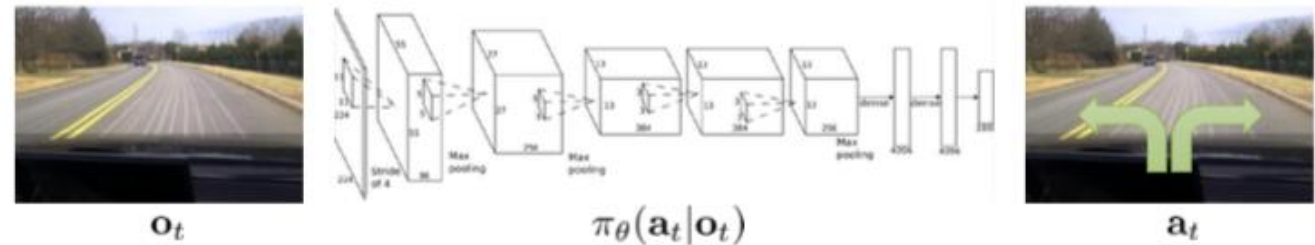
r – reward function

$r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$



More on POMDPs later in the course

Rewards are Obtained After Taking Actions



which action is better or worse?

$r(\mathbf{s}, \mathbf{a})$: reward function

tells us which states and actions are better

$\mathbf{s}$, $\mathbf{a}$, $r(\mathbf{s}, \mathbf{a})$, and $p(\mathbf{s}' | \mathbf{s}, \mathbf{a})$ define
Markov decision process

Rewards are an immediate evaluation
of the action executed.



high reward



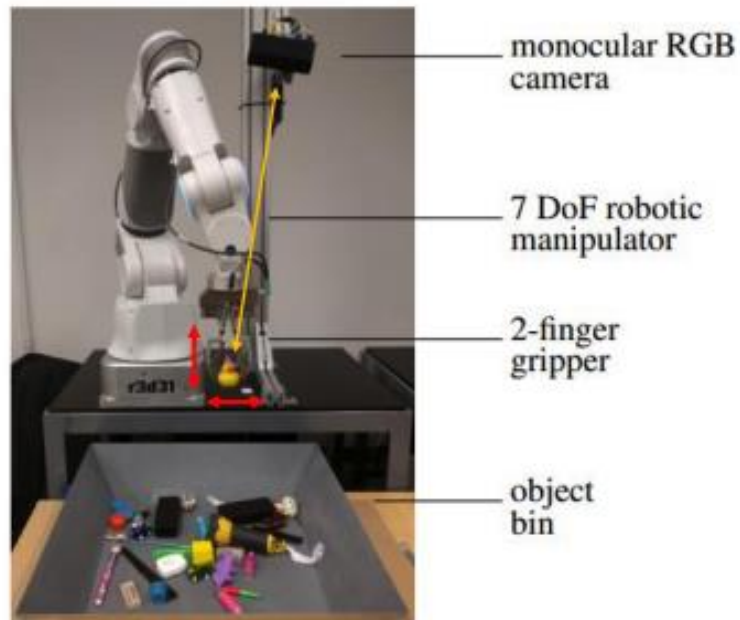
low reward

The RL setting: one collects (state, action, reward) data. Using the accumulated data one learns a good policy.

Example: Grasping



Example: Grasping



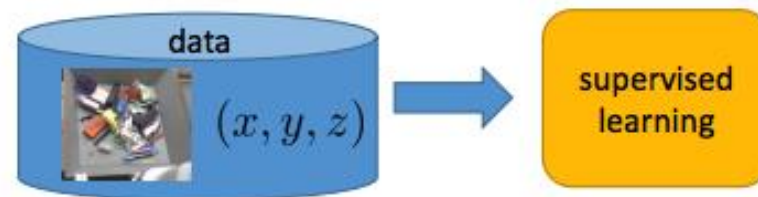
Option 1:

Understand the problem, design a solution

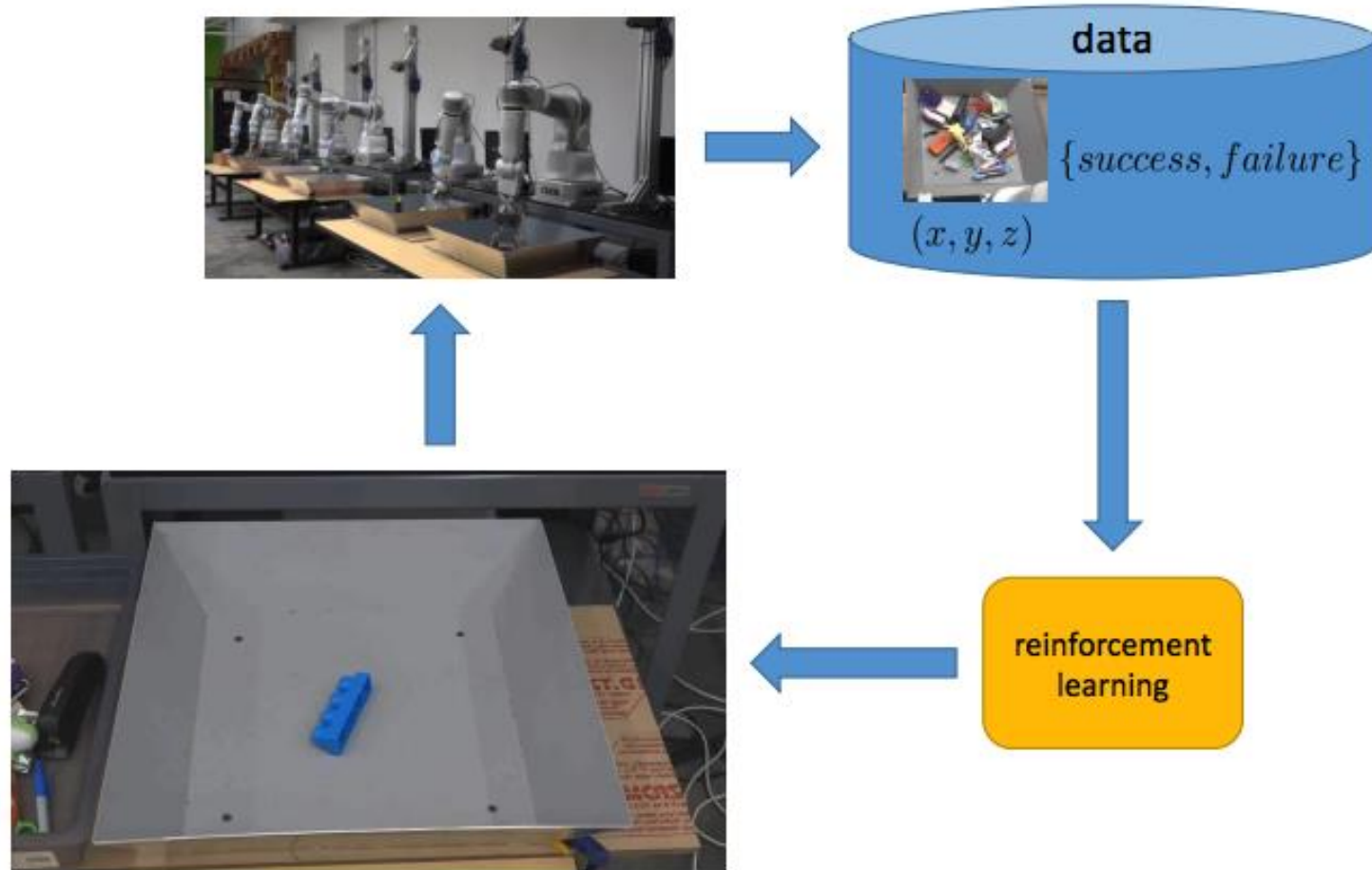


Option 2:

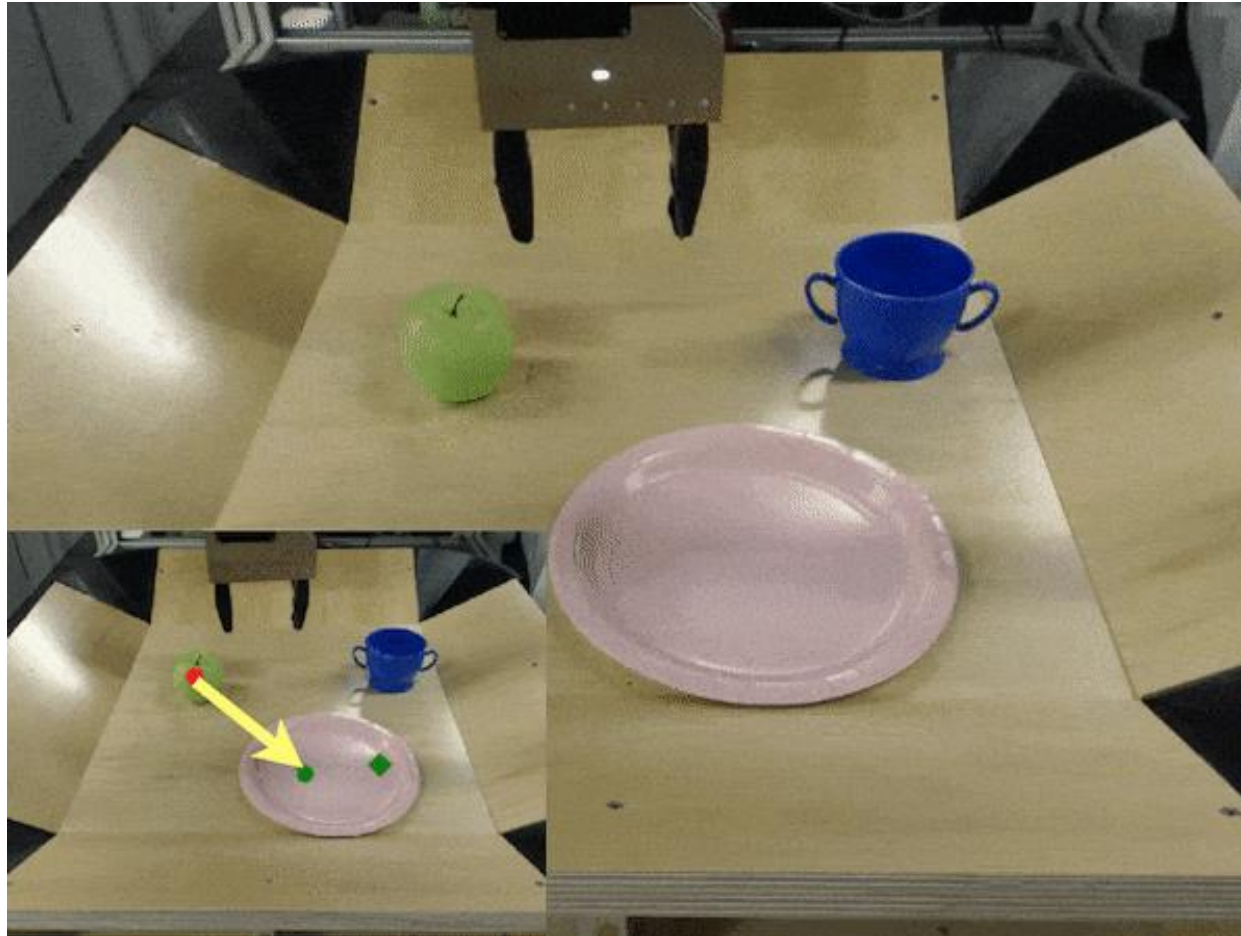
Set it up as a machine learning problem



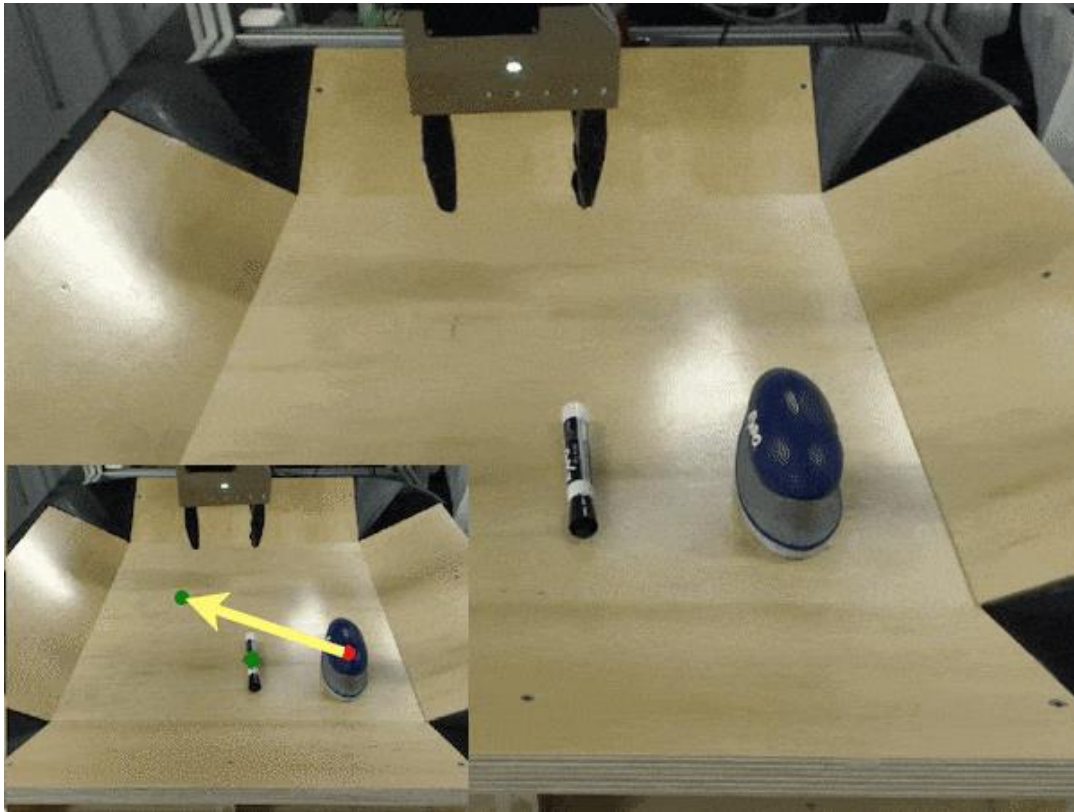
Example: Grasping



Examples: Learning to Manipulate



Examples: Learning to Manipulate



Examples: Learning to Walk



Initial

Examples: Learning to Walk



Training

Examples: Learning to Walk



Finished

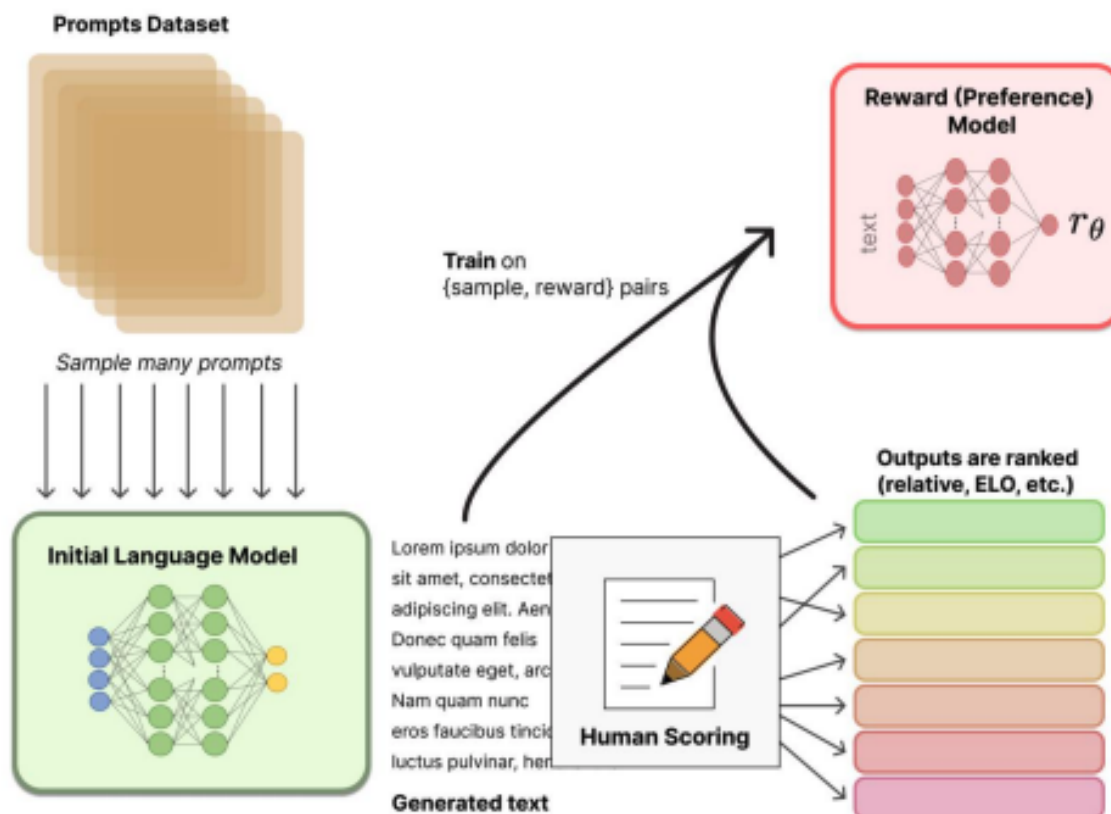
Examples: Learning to Perform Complex Skills



Given a single demonstration on a simple pipe, CBN-IRL can generalize to more complex environments.

Inverse RL a form of reinforcement learning

RL with Language Models



LE

Can you explain to me how RL with human feedback works for language models like I am five?



Sure! So, imagine you have a robot named Rufus who wants to learn how to talk like a human. Rufus has a language model that helps him understand words and sentences.

First, Rufus will say something using his language model. For example, he might say "I am a robot."

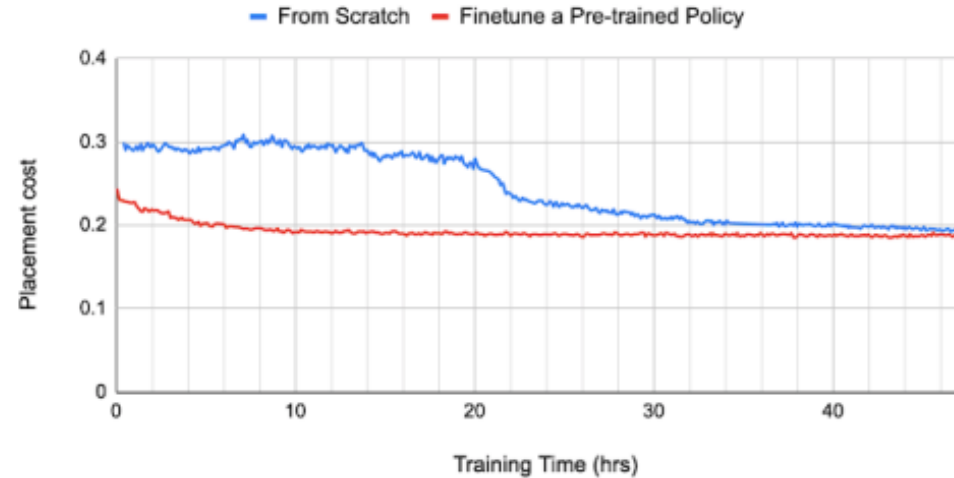
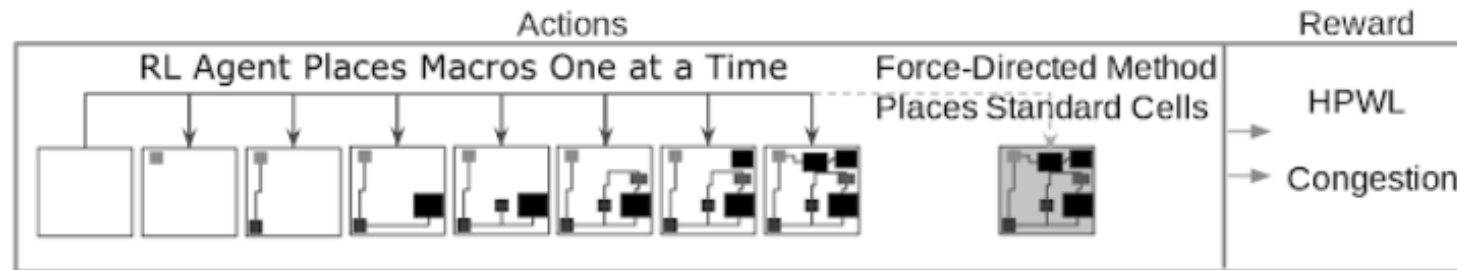
Then, a human will listen to what Rufus said and give him feedback on whether it sounded like a natural sentence a human would say. The human might say, "That's not quite right, Rufus. Humans don't usually say 'I am a robot.' They might say 'I'm a robot' or 'I am a machine.'"

Rufus will take this feedback and use it to update his language model. He will try to say the sentence again, using the new information he received from the human. This time, he might say "I'm a robot."

The human will listen again and give Rufus more feedback. This process will continue until Rufus can say sentences that sound natural to a human.

Over time, Rufus will learn how to talk like a human thanks to the feedback he receives from humans. This is how language models can be improved using RL with human feedback.

RL for Chip Design



Source: <https://ai.googleblog.com/2020/04/chip-design-with-deep-reinforcement.html>

Main Takeaways

- MDPs assumed that we know an a-priori model of the domain
 - In which case, we can derive the optimal policy. Then the agent can simply follow it.
- What happens when?
 - We do not have the following:
 - Transition function (*if I take an action which state I will end up in?*)
 - Reward function (*when I take a transition is it good or bad?*)
- The agent must interact/explore the environment
 - Learning from experience
 - It observes transitions and rewards as a function of actions
 - Learns the optimal policy from the experience data it collects.